

```
1 apps/api/main.py
2 """Development entry point for the API server."""
3 // empty line
4 from __future__ import annotations
5 // empty line
6 from wsgiref.simple_server import make_server
7 // empty line
8 from app.core.config import settings
9 from app.main import create_application
10 // empty line
11 // empty line
12 def main() -> None:
13     application = create_application()
14     with make_server(settings.host, settings.port,
15 application) as server:
16         print(f"Servicing {settings.app_name} at http://{settings.host}:
17 {settings.port}")
18         server.serve_forever()
19 // empty line
20 // empty line
21 if __name__ == "__main__":
22     main()
23 // section delimiter
24 apps/api/app/main.py
25 """WSGI application factory."""
26 // empty line
27 from __future__ import annotations
28 // empty line
29 from .core.database import init_database
30 from .core.seed import seed_database
31 from .core.routing import Application, Router
32 from .modules import audit, auth, dashboard, files, finance, iam,
33 inventory, masterdata, notifications, procurement, sales, workflows
34 // empty line
35 // empty line
36 def create_application() -> Application:
37     init_database()
38     seed_database()
39     router = Router()
40     auth.register(router)
41     iam.register(router)
42     masterdata.register(router)
43     workflows.register(router)
44     sales.register(router)
45     procurement.register(router)
46     inventory.register(router)
47     finance.register(router)
48     dashboard.register(router)
49     notifications.register(router)
50     files.register(router)
```

```

51         audit.register(router)
52         return Application(router)
53     // section delimiter
54     apps/api/app/core/config.py
55     """Application configuration."""
56     // empty line
57     from __future__ import annotations
58     // empty line
59     import os
60     from dataclasses import dataclass
61     from pathlib import Path
62     // empty line
63     // empty line
64     REPO_ROOT = Path(__file__).resolve().parents[4]
65     // empty line
66     // empty line
67     @dataclass(frozen=True)
68     class Settings:
69         """Runtime settings."""
70     // empty line
71     app_name: str = "企业运营全流程数字化管控平台"
72     api_prefix: str = "/api/v1"
73     host: str = os.environ.get("APP_HOST", "127.0.0.1")
74     port: int = int(os.environ.get("APP_PORT", "8000"))
75     database_path: Path = Path(
76         os.environ.get(
77             "APP_DATABASE_PATH",
78             str(REPO_ROOT / "apps" / "api" / "data.sqlite3"),
79         )
80     )
81     migrations_dir: Path = REPO_ROOT / "apps" / "api" / "migrations"
82     web_root: Path = REPO_ROOT / "apps" / "web"
83     access_token_minutes: int = int(os.environ.get(
84         ("APP_ACCESS_TOKEN_MINUTES", "60"))
85     )
86     refresh_token_minutes: int = int(os.environ.get(
87         ("APP_REFRESH_TOKEN_MINUTES", "1440"))
88     )
89     password_lock_minutes: int = int(os.environ.get(
90         ("APP_PASSWORD_LOCK_MINUTES", "15"))
91     )
92     password_fail_threshold: int = int(os.environ.get(
93         ("APP_PASSWORD_FAIL_THRESHOLD", "5"))
94     )
95     token_secret: str = os.environ.get(
96         "APP_TOKEN_SECRET",
97         "local-demo-secret-key-change-before-production",
98     )
99     attachment_max_size: int = int(os.environ.get(
100         ("APP_ATTACHMENT_MAX_SIZE", "10485760"))
101     )
102     enable_hard_budget_control: bool = os.environ.get(
103         "APP_ENABLE_HARD_BUDGET_CONTROL",
104         "1",
105     ) == "1"

```

```
101 // empty line
102 // empty line
103 settings = Settings()
104 // section delimiter
105 apps/api/app/core/database.py
106 """SQLite database helpers and migration runner."""
107 // empty line
108 from __future__ import annotations
109 // empty line
110 import sqlite3
111 from contextlib import contextmanager
112 from pathlib import Path
113 from typing import Any, Iterator
114 // empty line
115 from .config import settings
116 // empty line
117 // empty line
118 def ensure_parent_directory(database_path: Path) -> None:
119     database_path.parent.mkdir(parents=True, exist_ok=True)
120 // empty line
121 // empty line
122 def get_connection() -> sqlite3.Connection:
123     """Return a configured SQLite connection."""
124 // empty line
125     ensure_parent_directory(settings.database_path)
126     connection = sqlite3.connect(str(settings.database_path))
127     connection.row_factory = sqlite3.Row
128     connection.execute("PRAGMA foreign_keys = ON")
129     return connection
130 // empty line
131 // empty line
132 @contextmanager
133 def transaction() -> Iterator[sqlite3.Connection]:
134     """Provide a commit or rollback guarded transaction."""
135 // empty line
136     connection = get_connection()
137     try:
138         yield connection
139         connection.commit()
140     except Exception:
141         connection.rollback()
142         raise
143     finally:
144         connection.close()
145 // empty line
146 // empty line
147 def row_to_dict(row: sqlite3.Row | None) -> dict[str, Any] | None:
148     if row is None:
149         return None
150     return {key: row[key] for key in row.keys()}
```

```

151 // empty line
152 // empty line
153 def fetch_one(
154     connection: sqlite3.Connection,
155     sql: str,
156     params: tuple[Any, ...] = (),
157 ) -> dict[str, Any] | None:
158     row = connection.execute(sql, params).fetchone()
159     return row_to_dict(row)
160 // empty line
161 // empty line
162 def fetch_all(
163     connection: sqlite3.Connection,
164     sql: str,
165     params: tuple[Any, ...] = (),
166 ) -> list[dict[str, Any]]:
167     rows = connection.execute(sql, params).fetchall()
168     return [row_to_dict(row) for row in rows if row is not None]
169 // empty line
170 // empty line
171 def execute(
172     connection: sqlite3.Connection,
173     sql: str,
174     params: tuple[Any, ...] = (),
175 ) -> None:
176     connection.execute(sql, params)
177 // empty line
178 // empty line
179 def init_database() -> None:
180     """Run SQL migrations in lexical order."""
181 // empty line
182     with transaction() as connection:
183         connection.execute(
184             """
185             CREATE TABLE IF NOT EXISTS schema_migrations (
186                 version TEXT PRIMARY KEY,
187                 applied_at TEXT NOT NULL
188             )
189             """
190         )
191         applied_rows = connection.execute
192         ("SELECT version FROM schema_migrations").fetchall()
193         applied_versions = {row[0] for row in applied_rows}
194         migration_files = sorted(settings.migrations_dir.glob("*.sql"))
195         for migration_file in migration_files:
196             version = migration_file.name
197             if version in applied_versions:
198                 continue
199             sql = migration_file.read_text(encoding="utf-8")
200             connection.executescript(sql)

```

```

201         connection.execute(
202             "INSERT INTO schema_migrations(version,
203 applied_at) VALUES (?, datetime('now'))",
204             (version,),
205         )
206 // section delimiter
207 apps/api/app/core/exceptions.py
208 """Custom exceptions and response mapping."""
209 // empty line
210 from __future__ import annotations
211 // empty line
212 // empty line
213 class AppError(Exception):
214     """Base application error."""
215 // empty line
216     def __init__(self, status_code: int, code: str,
217 message: str) -> None:
218         super().__init__(message)
219         self.status_code = status_code
220         self.code = code
221         self.message = message
222 // empty line
223 // empty line
224 class ValidationError(AppError):
225     def __init__(self, message: str) -> None:
226         super().__init__(400, "REQ_400_01", message)
227 // empty line
228 // empty line
229 class UnauthorizedError(AppError):
230     def __init__(self, message: str = "未登录或令牌无效") -> None:
231         super().__init__(401, "AUTH_401_01", message)
232 // empty line
233 // empty line
234 class ForbiddenError(AppError):
235     def __init__(self, message: str = "权限不足") -> None:
236         super().__init__(403, "AUTH_403_01", message)
237 // empty line
238 // empty line
239 class NotFoundError(AppError):
240     def __init__(self, message: str) -> None:
241         super().__init__(404, "REQ_404_01", message)
242 // empty line
243 // empty line
244 class ConflictError(AppError):
245     def __init__(self, message: str, code: str = "REQ_409_01") -> None:
246         super().__init__(409, code, message)
247 // empty line
248 // empty line
249 class FileTooLargeError(AppError):
250     def __init__(self, message: str = "附件大小超限") -> None:

```

```

251         super().__init__(413, "FILE_413_01", message)
252     // section delimiter
253     apps/api/app/core/responses.py
254     """Helpers for JSON and text responses."""
255     // empty line
256     from __future__ import annotations
257     // empty line
258     import json
259     from typing import Any
260     // empty line
261     // empty line
262     def json_response(
263         data: Any,
264         request_id: str,
265         *,
266         code: str = "OK",
267         message: str = "成功",
268         status_code: int = 200,
269     ) -> tuple[int, list[tuple[str, str]], bytes]:
270         body = json.dumps(
271             {
272                 "code": code,
273                 "message": message,
274                 "requestId": request_id,
275                 "data": data,
276             },
277             ensure_ascii=False,
278         ).encode("utf-8")
279         headers = [
280             ("Content-Type", "application/json; charset=utf-8"),
281             ("Content-Length", str(len(body))),
282         ]
283         return status_code, headers, body
284     // empty line
285     // empty line
286     def page_response(
287         items: list[Any],
288         page: int,
289         page_size: int,
290         total: int,
291         request_id: str,
292     ) -> tuple[int, list[tuple[str, str]], bytes]:
293         return json_response(
294             {
295                 "items": items,
296                 "page": page,
297                 "pageSize": page_size,
298                 "total": total,
299             },
300             request_id,

```

```

301         )
302     // empty line
303     // empty line
304     def text_response(
305         text: str,
306         *,
307         status_code: int = 200,
308         content_type: str = "text/plain; charset=utf-8",
309     ) -> tuple[int, list[tuple[str, str]], bytes]:
310         body = text.encode("utf-8")
311         headers = [
312             ("Content-Type", content_type),
313             ("Content-Length", str(len(body))),
314         ]
315         return status_code, headers, body
316     // section delimiter
317     apps/api/app/core/routing.py
318     """Minimal WSGI router and request context."""
319     // empty line
320     from __future__ import annotations
321     // empty line
322     import json
323     import mimetypes
324     import re
325     import sqlite3
326     import traceback
327     from dataclasses import dataclass, field
328     from typing import Any, Callable
329     from urllib.parse import parse_qs
330     // empty line
331     from .config import settings
332     from .database import get_connection
333     from .exceptions import AppError, NotFoundError, UnauthorizedError
334     from .responses import json_response
335     from .security import decode_signed_token
336     // empty line
337     // empty line
338     Handler = Callable[["RequestContext"], tuple[int, list[tuple[str, str]],
339     bytes]]
340     // empty line
341     // empty line
342     @dataclass
343     class Route:
344         method: str
345         path: str
346         handler: Handler
347         auth_required: bool = True
348         permission: str | None = None
349         compiled: re.Pattern[str] = field(init=False)
350     // empty line

```

```

351     def __post_init__(self) -> None:
352         pattern = re.sub(r"{{[a-zA-Z_][a-zA-Z0-9_]*}}", r"(?P<\1>
353 [^/]+)", self.path)
354         self.compiled = re.compile(f"^{pattern}$")
355 // empty line
356 // empty line
357 @dataclass
358 class RequestContext:
359     method: str
360     path: str
361     headers: dict[str, str]
362     query: dict[str, Any]
363     body: dict[str, Any]
364     raw_body: bytes
365     path_params: dict[str, str]
366     request_id: str
367     db: Any
368     user: dict[str, Any] | None = None
369 // empty line
370     def header(self, name: str) -> str | None:
371         return self.headers.get(name.lower())
372 // empty line
373 // empty line
374 class Router:
375     def __init__(self) -> None:
376         self.routes: list[Route] = []
377 // empty line
378     def add(
379         self,
380         method: str,
381         path: str,
382         handler: Handler,
383         *,
384         auth_required: bool = True,
385         permission: str | None = None,
386     ) -> None:
387         self.routes.append(
388             Route(
389                 method=method.upper(),
390                 path=path,
391                 handler=handler,
392                 auth_required=auth_required,
393                 permission=permission,
394             )
395         )
396 // empty line
397     def match(self, method: str, path: str) -> tuple[Route, dict[str,
398 str]]:
399         for route in self.routes:
400             if route.method != method.upper():

```

```

401         continue
402         match = route.compiled.match(path)
403         if match:
404             return route, match.groupdict()
405         raise NotFoundError(f"接口不存在: {path}")
406 // empty line
407 // empty line
408 def parse_query(envIRON: dict[str, Any]) -> dict[str, Any]:
409     parsed = parse_qs(envIRON.get("QUERY_STRING", ""),
410 keep_blank_values=True)
411     return {key: values[0] if len(values) == 1 else values for key,
412 values in parsed.items()}
413 // empty line
414 // empty line
415 def parse_body(envIRON: dict[str, Any]) -> tuple[bytes, dict[str, Any]]:
416     try:
417         body_size = int(envIRON.get("CONTENT_LENGTH", "0") or 0)
418     except ValueError:
419         body_size = 0
420     raw_body = envIRON["wsgi.input"].read
421 (body_size) if body_size > 0 else b""
422     if not raw_body:
423         return raw_body, {}
424     if "application/json" in envIRON.get("CONTENT_TYPE", ""):
425         return raw_body, json.loads(raw_body.decode("utf-8"))
426     return raw_body, {}
427 // empty line
428 // empty line
429 def collect_headers(envIRON: dict[str, Any]) -> dict[str, str]:
430     headers: dict[str, str] = {}
431     for key, value in envIRON.items():
432         if key.startswith("HTTP_"):
433             headers[key[5:].replace("_", "-").lower()] = value
434     if envIRON.get("CONTENT_TYPE"):
435         headers["content-type"] = envIRON["CONTENT_TYPE"]
436     return headers
437 // empty line
438 // empty line
439 def load_user_from_token(token: str, connection: Any) -> dict[str, Any]:
440     payload = decode_signed_token(token)
441     row = connection.execute(
442         "SELECT * FROM users WHERE id = ? AND status = 'ACTIVE'",
443         (payload["sub"],),
444     ).fetchone()
445     if row is None:
446         raise UnauthorizedError()
447     user = {key: row[key] for key in row.keys()}
448     user["token_payload"] = payload
449     return user
450 // empty line

```

```

451 // empty line
452 def authenticate(route: Route, headers: dict[str, str],
453 connection: Any) -> dict[str, Any] | None:
454     if not route.auth_required:
455         return None
456     authorization = headers.get("authorization", "")
457     if not authorization.startswith("Bearer "):
458         raise UnauthorizedError()
459     token = authorization.split(" ", 1)[1]
460     return load_user_from_token(token, connection)
461 // empty line
462 // empty line
463 def status_text(status_code: int) -> str:
464     mapping = {
465         200: "200 OK",
466         201: "201 Created",
467         400: "400 Bad Request",
468         401: "401 Unauthorized",
469         403: "403 Forbidden",
470         404: "404 Not Found",
471         409: "409 Conflict",
472         413: "413 Payload Too Large",
473         500: "500 Internal Server Error",
474     }
475     return mapping.get(status_code, f"{status_code} OK")
476 // empty line
477 // empty line
478 def serve_static(path: str) -> tuple[int, list[tuple[str, str]], bytes]:
479     safe_path = path.lstrip("/") or "index.html"
480     candidate = (settings.web_root / safe_path).resolve()
481     if not str(candidate).startswith(str(settings.web_root.resolve
482 ())) or not candidate.exists():
483         candidate = settings.web_root / "index.html"
484     content = candidate.read_bytes()
485     mime_type, _ = mimetypes.guess_type(str(candidate))
486     return 200, [
487         ("Content-Type", f"{mime_type or 'text/html'}; charset=utf-8"),
488         ("Content-Length", str(len(content))),
489     ], content
490 // empty line
491 // empty line
492 class Application:
493     def __init__(self, router: Router) -> None:
494         self.router = router
495 // empty line
496     def __call__(self, environ: dict[str, Any],
497 start_response: Callable[..., Any]) -> list[bytes]:
498         path = environ.get("PATH_INFO", "/")
499         if not path.startswith(settings.api_prefix):
500             status_code, headers, body = serve_static(path)

```

```

501         start_response(status_text(status_code), headers)
502         return [body]
503     // empty line
504     request_id = environ.get("HTTP_X_REQUEST_ID") or f"req_{abs
505 (hash(path + environ.get('REQUEST_METHOD', 'GET')))}"
506     method = environ.get("REQUEST_METHOD", "GET").upper()
507     query = parse_query(environ)
508     headers = collect_headers(environ)
509     raw_body, body = parse_body(environ)
510     connection = get_connection()
511     try:
512         route, path_params = self.router.match(method, path)
513         user = authenticate(route, headers, connection)
514         context = RequestContext(
515             method=method,
516             path=path,
517             headers=headers,
518             query=query,
519             body=body,
520             raw_body=raw_body,
521             path_params=path_params,
522             request_id=request_id,
523             db=connection,
524             user=user,
525         )
526         status_code, response_headers, response_body =
527 route.handler(context)
528         connection.commit()
529     except AppError as exc:
530         connection.rollback()
531         status_code, response_headers, response_body =
532 json_response(
533     {},
534     request_id,
535     code=exc.code,
536     message=exc.message,
537     status_code=exc.status_code,
538 )
539     except sqlite3.IntegrityError as exc:
540         connection.rollback()
541         traceback.print_exc()
542         status_code, response_headers, response_body =
543 json_response(
544     {},
545     request_id,
546     code="REQ_409_01",
547     message=f"数据约束冲突: {exc}",
548     status_code=409,
549 )
550     except Exception:

```

```
551         connection.rollback()
552         traceback.print_exc()
553         status_code, response_headers, response_body =
554 json_response(
555     {},
556     request_id,
557     code="SYS_500_01",
558     message="系统内部错误",
559     status_code=500,
560 )
561 finally:
562     connection.close()
563     start_response(status_text(status_code), response_headers)
564     return [response_body]
565 // section delimiter
566 apps/api/app/core/security.py
567 """Token and password helpers."""
568 // empty line
569 from __future__ import annotations
570 // empty line
571 import base64
572 import hashlib
573 import hmac
574 import json
575 import os
576 import uuid
577 from datetime import datetime, timedelta, timezone
578 from typing import Any
579 // empty line
580 from .config import settings
581 from .exceptions import UnauthorizedError
582 // empty line
583 // empty line
584 def utcnow() -> datetime:
585     return datetime.now(timezone.utc)
586 // empty line
587 // empty line
588 def iso_now() -> str:
589     return utcnow().replace(microsecond=0).isoformat()
590 // empty line
591 // empty line
592 def base64url_encode(raw: bytes) -> str:
593     return base64.urlsafe_b64encode(raw).rstrip(b"=").decode("ascii")
594 // empty line
595 // empty line
596 def base64url_decode(raw: str) -> bytes:
597     padding = "=" * (-len(raw) % 4)
598     return base64.urlsafe_b64decode((raw + padding).encode("ascii"))
599 // empty line
600 // empty line
```

```

601 def generate_salt() -> str:
602     return base64url_encode(os.urandom(16))
603 // empty line
604 // empty line
605 def hash_password(password: str, salt: str) -> str:
606     salt_bytes = base64url_decode(salt)
607     derived = hashlib.scrypt(
608         password.encode("utf-8"),
609         salt=salt_bytes,
610         n=2**14,
611         r=8,
612         p=1,
613         dklen=32,
614     )
615     return derived.hex()
616 // empty line
617 // empty line
618 def verify_password(password: str, salt: str,
619 expected_hash: str) -> bool:
620     return hmac.compare_digest(hash_password(password, salt),
621 expected_hash)
622 // empty line
623 // empty line
624 def create_signed_token(payload: dict[str, Any],
625 expires_minutes: int) -> str:
626     header = {"alg": "HS256", "typ": "JWT"}
627     payload_data = dict(payload)
628     payload_data["exp"] = int((utcnow() + timedelta
629 (minutes=expires_minutes)).timestamp())
630     payload_data.setdefault("jti", uuid.uuid4().hex)
631     header_b64 = base64url_encode(json.dumps(header, separators=(",",
632 ":")).encode("utf-8"))
633     payload_b64 = base64url_encode(
634         json.dumps(payload_data, separators=(",", ":"),
635 ensure_ascii=False).encode("utf-8"))
636 )
637     signing_input = f"{header_b64}.{payload_b64}".encode("ascii")
638     signature = hmac.new(
639         settings.token_secret.encode("utf-8"),
640         signing_input,
641         hashlib.sha256,
642     ).digest()
643     return f"{header_b64}.{payload_b64}.{base64url_encode(signature)}"
644 // empty line
645 // empty line
646 def decode_signed_token(token: str) -> dict[str, Any]:
647     try:
648         header_b64, payload_b64, signature_b64 = token.split(".")
649         except ValueError as exc:
650             raise UnauthorizedError() from exc

```

```

651     signing_input = f"{header_b64}.{payload_b64}".encode("ascii")
652     expected_signature = hmac.new(
653         settings.token_secret.encode("utf-8"),
654         signing_input,
655         hashlib.sha256,
656     ).digest()
657     if not hmac.compare_digest(base64url_decode(signature_b64),
658 expected_signature):
659         raise UnauthorizedError()
660     payload = json.loads(base64url_decode(payload_b64).decode("utf-8"))
661     if int(payload["exp"]) < int(utcnow().timestamp()):
662         raise UnauthorizedError("令牌已过期")
663     return payload
664 // section delimiter
665 apps/api/app/core/seed.py
666 """Seed demo data for local development and tests."""
667 // empty line
668 from __future__ import annotations
669 // empty line
670 from datetime import datetime, timedelta
671 from typing import Any
672 // empty line
673 from .database import init_database, transaction
674 from .security import generate_salt, hash_password, iso_now
675 from .modules.common import generate_document_no, generate_id,
676 json_dumps
677 from ..modules.constants import (
678     BIZ_CONTRACT,
679     BIZ_EXPENSE,
680     BIZ_PROCUREMENT_REQUEST,
681     ROLE_AUDITOR,
682     ROLE_FINANCE,
683     ROLE_OPERATIONS_DIRECTOR,
684     ROLE_PERMISSION_MAP,
685     ROLE_PROCUREMENT,
686     ROLE_SALES,
687     ROLE_SCOPE_MAP,
688     ROLE_SYSTEM_ADMIN,
689     ROLE_WAREHOUSE,
690     STATUS_ACTIVE,
691     STATUS_APPROVED,
692     STATUS_COMPLETED,
693     STATUS_EFFECTIVE,
694     STATUS_OPEN,
695     STATUS_PARTIAL,
696     STATUS_PENDING,
697     WORKFLOW_STATUS_DONE,
698     WORKFLOW_STATUS_OPEN,
699 )
700 // empty line

```

```

701 // empty line
702 def _clear_all(connection: Any) -> None:
703     tables = [
704         "approval_tasks",
705         "workflow_instances",
706         "workflow_templates",
707         "payment_requests",
708         "receipt_records",
709         "expense_claims",
710         "stock_balances",
711         "inventory_txns",
712         "purchase_orders",
713         "procurement_requests",
714         "contracts",
715         "sales_orders",
716         "notifications",
717         "file_assets",
718         "audit_logs",
719         "budgets",
720         "warehouses",
721         "projects",
722         "item_master",
723         "suppliers",
724         "customers",
725         "cost_centers",
726         "system_configs",
727         "users",
728         "positions",
729         "org_units",
730         "roles",
731         "refresh_tokens",
732         "idempotency_keys",
733     ]
734     for table in tables:
735         connection.execute(f"DELETE FROM {table}")
736 // empty line
737 // empty line
738 def _insert_role(connection: Any, code: str, name: str) -> str:
739     role_id = generate_id("role")
740     now = iso_now()
741     connection.execute(
742         """
743         INSERT INTO roles(
744             id, code, name, permissions_json, data_scope_type, status,
745             created_at, created_by, updated_at, updated_by, version
746         ) VALUES (?, ?, ?, ?, ?, ?, ?, 'seed', ?, 'seed', 1)
747         """,
748         (
749             role_id,
750             code,

```

```

751         name,
752         json_dumps(ROLE_PERMISSION_MAP[code]),
753         ROLE_SCOPE_MAP[code],
754         STATUS_ACTIVE,
755         now,
756         now,
757     ),
758 )
759     return role_id
760 // empty line
761 // empty line
762 def _insert_org(connection: Any, parent_id: str | None, name: str,
763 unit_type: str) -> str:
764     org_id = generate_id("org")
765     now = iso_now()
766     connection.execute(
767         """
768         INSERT INTO org_units(
769             id, parent_id, name, unit_type, manager_user_id, status,
770             created_at, created_by, updated_at, updated_by, version
771         ) VALUES (?, ?, ?, ?, NULL, ?, ?, 'seed', ?, 'seed', 1)
772         """,
773         (org_id, parent_id, name, unit_type, STATUS_ACTIVE, now, now),
774     )
775     return org_id
776 // empty line
777 // empty line
778 def _insert_position(connection: Any, org_unit_id: str,
779 name: str) -> str:
780     position_id = generate_id("pos")
781     now = iso_now()
782     connection.execute(
783         """
784         INSERT INTO positions(
785             id, org_unit_id, name, status, created_at, created_by,
786             updated_at, updated_by, version
787         ) VALUES (?, ?, ?, ?, ?, 'seed', ?, 'seed', 1)
788         """,
789         (position_id, org_unit_id, name, STATUS_ACTIVE, now, now),
790     )
791     return position_id
792 // empty line
793 // empty line
794 def _insert_user(
795     connection: Any,
796     *,
797     username: str,
798     display_name: str,
799     org_unit_id: str,
800     position_id: str,

```

```

801         roles: list[str],
802         email: str,
803         mobile: str,
804         password: str = "admin123",
805     ) -> str:
806         user_id = generate_id("user")
807         salt = generate_salt()
808         now = iso_now()
809         connection.execute(
810             """
811             INSERT INTO users(
812                 id, username, password_hash, password_salt, display_name,
813                 email, mobile,
814                 org_unit_id, position_id, role_codes_json, status,
815                 failed_login_count, locked_until, created_at, created_by,
816                 updated_at, updated_by, version
817             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, 0, NULL, ?, 'seed',
818             ?, 'seed', 1)
819             """
820             (
821                 user_id,
822                 username,
823                 hash_password(password, salt),
824                 salt,
825                 display_name,
826                 email,
827                 mobile,
828                 org_unit_id,
829                 position_id,
830                 json.dumps(roles),
831                 STATUS_ACTIVE,
832                 now,
833                 now,
834             ),
835         )
836         return user_id
837     // empty line
838     // empty line
839     def seed_database(force: bool = False) -> dict[str, int]:
840         init_database()
841         with transaction() as connection:
842             existing_users = connection.execute("SELECT COUNT
843             (1) AS total FROM users").fetchone()["total"]
844             if existing_users and not force:
845                 return {"users": existing_users}
846             if force:
847                 _clear_all(connection)
848     // empty line
849     role_names = {
850         ROLE_SYSTEM_ADMIN: "系统管理员",

```

```

851     ROLE_OPERATIONS_DIRECTOR: "运营总监",
852     ROLE_SALES: "销售专员",
853     ROLE_PROCUREMENT: "采购专员",
854     ROLE_WAREHOUSE: "仓库管理员",
855     ROLE_FINANCE: "财务专员",
856     ROLE_AUDITOR: "审计人员",
857 }
858 for code, name in role_names.items():
859     _insert_role(connection, code, name)
860 // empty line
861 org_hq = _insert_org(connection, None, "集团总部", "COMPANY")
862 org_north = _insert_org(connection, org_hq, "北方事业部", "DIVISION")
863 org_south = _insert_org(connection, org_hq, "南方事业部", "DIVISION")
864 org_north_sales = _insert_org(connection, org_north, "北方销售部",
865 "DEPARTMENT")
866 org_north_proc = _insert_org(connection, org_north, "北方采购部",
867 "DEPARTMENT")
868 org_north_fin = _insert_org(connection, org_north, "北方财务部",
869 "DEPARTMENT")
870 org_north_wh = _insert_org(connection, org_north, "北方仓配中心",
871 "DEPARTMENT")
872 org_south_sales = _insert_org(connection, org_south, "南方销售部",
873 "DEPARTMENT")
874 org_south_proc = _insert_org(connection, org_south, "南方采购部",
875 "DEPARTMENT")
876 org_south_fin = _insert_org(connection, org_south, "南方财务部",
877 "DEPARTMENT")
878 org_south_wh = _insert_org(connection, org_south, "南方仓配中心",
879 "DEPARTMENT")
880 // empty line
881 pos_admin = _insert_position(connection, org_hq, "平台管理员")
882 pos_director = _insert_position(connection, org_north, "运营总监")
883 pos_sales = _insert_position(connection, org_north_sales,
884 "销售经理")
885 pos_proc = _insert_position(connection, org_north_proc, "采购经理")
886 pos_fin = _insert_position(connection, org_north_fin, "财务经理")
887 pos_wh = _insert_position(connection, org_north_wh, "仓储主管")
888 pos_auditor = _insert_position(connection, org_hq, "审计经理")
889 // empty line
890 users = {
891     "admin": _insert_user(
892         connection,
893         username="admin",
894         display_name="平台管理员",
895         org_unit_id=org_hq,
896         position_id=pos_admin,
897         roles=[ROLE_SYSTEM_ADMIN, ROLE_AUDITOR],
898         email="admin@example.com",
899         mobile="13800000000",
900     ),

```

```
901 "director": _insert_user(  
902     connection,  
903     username="director",  
904     display_name="运营总监",  
905     org_unit_id=org_north,  
906     position_id=pos_director,  
907     roles=[ROLE_OPERATIONS_DIRECTOR],  
908     email="director@example.com",  
909     mobile="13800000001",  
910 ),  
911 "sales": _insert_user(  
912     connection,  
913     username="sales01",  
914     display_name="销售专员",  
915     org_unit_id=org_north_sales,  
916     position_id=pos_sales,  
917     roles=[ROLE_SALES],  
918     email="sales01@example.com",  
919     mobile="13800000002",  
920 ),  
921 "procurement": _insert_user(  
922     connection,  
923     username="proc01",  
924     display_name="采购专员",  
925     org_unit_id=org_north_proc,  
926     position_id=pos_proc,  
927     roles=[ROLE_PROCUREMENT],  
928     email="proc01@example.com",  
929     mobile="13800000003",  
930 ),  
931 "finance": _insert_user(  
932     connection,  
933     username="fin01",  
934     display_name="财务专员",  
935     org_unit_id=org_north_fin,  
936     position_id=pos_fin,  
937     roles=[ROLE_FINANCE],  
938     email="fin01@example.com",  
939     mobile="13800000004",  
940 ),  
941 "warehouse": _insert_user(  
942     connection,  
943     username="wh01",  
944     display_name="仓储专员",  
945     org_unit_id=org_north_wh,  
946     position_id=pos_wh,  
947     roles=[ROLE_WAREHOUSE],  
948     email="wh01@example.com",  
949     mobile="13800000005",  
950 ),
```

```

951         "auditor": _insert_user(
952             connection,
953             username="audit01",
954             display_name="审计专员",
955             org_unit_id=org_hq,
956             position_id=pos_auditor,
957             roles=[ROLE_AUDITOR],
958             email="audit01@example.com",
959             mobile="13800000006",
960         ),
961     }
962 // empty line
963     now = iso_now()
964     configs = [
965         ("oidc.enabled", "false", "是否启用 OIDC 单点登录"),
966         ("attachment.maxSize", "10485760", "附件大小限制"),
967         ("workflow.timeoutHours", "24", "审批超时小时数"),
968         ("budget.controlMode", "HARD", "预算控制模式"),
969     ]
970     for key, value, desc in configs:
971         connection.execute(
972             """
973             INSERT INTO system_configs(
974                 id, config_key, config_value, description, status,
975                 created_at, created_by, updated_at, updated_by,
976                 version
977             ) VALUES (?, ?, ?, ?, ?, ?, 'seed', ?, 'seed', 1)
978             """
979             (generate_id("cfg"), key, value, desc, STATUS_ACTIVE,
980             now, now),
981         )
982 // empty line
983     cost_centers = []
984     org_sequence = [org_north_sales, org_north_proc, org_north_fin,
985 org_north_wh, org_south_sales, org_south_proc]
986     for index in range(1, 13):
987         cost_center_id = generate_id("cc")
988         cost_centers.append(cost_center_id)
989         connection.execute(
990             """
991             INSERT INTO cost_centers(
992                 id, code, name, org_unit_id, status,
993                 created_at, created_by, updated_at, updated_by,
994                 version
995             ) VALUES (?, ?, ?, ?, ?, ?, 'seed', ?, 'seed', 1)
996             """
997             (
998                 cost_center_id,
999                 f"CC{index:03d}",
1000                 f"成本中心{index:02d}",

```

```

1001         org_sequence[(index - 1) % len(org_sequence)],
1002         STATUS_ACTIVE,
1003         now,
1004         now,
1005     ),
1006 )
1007 // empty line
1008     customers = []
1009     for index in range(1, 26):
1010         customer_id = generate_id("cust")
1011         customers.append(customer_id)
1012         connection.execute(
1013             """
1014             INSERT INTO customers(
1015                 id, code, name, industry, owner_user_id,
1016                 contact_name, contact_phone, status,
1017                 created_at, created_by, updated_at, updated_by,
1018                 version
1019             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, 'seed', ?, 'seed',
1020             1)
1021             """
1022             (
1023                 customer_id,
1024                 f"CUST{index:03d}",
1025                 f"客户{index:03d}",
1026                 ["制造", "零售", "科技", "物流"][index % 4],
1027                 users["sales"],
1028                 f"联系人{index:02d}",
1029                 f"1390000{index:04d}",
1030                 STATUS_ACTIVE,
1031                 now,
1032                 now,
1033             ),
1034         )
1035 // empty line
1036     suppliers = []
1037     for index in range(1, 21):
1038         supplier_id = generate_id("sup")
1039         suppliers.append(supplier_id)
1040         connection.execute(
1041             """
1042             INSERT INTO suppliers(
1043                 id, code, name, category, rating, status,
1044                 created_at, created_by, updated_at, updated_by,
1045                 version
1046             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'seed', ?, 'seed', 1)
1047             """
1048             (
1049                 supplier_id,
1050                 f"SUP{index:03d}",

```

```

1051         f"供应商{index:03d}",
1052         ["办公", "物流", "服务", "设备"][index % 4],
1053         3 + index % 3,
1054         STATUS_ACTIVE,
1055         now,
1056         now,
1057     ),
1058 )
1059 // empty line
1060 items = []
1061 for index in range(1, 31):
1062     item_id = generate_id("item")
1063     items.append(item_id)
1064     connection.execute(
1065         """
1066         INSERT INTO item_master(
1067             id, code, name, item_type, uom, safety_stock,
1068             unit_price, status,
1069             created_at, created_by, updated_at, updated_by,
1070             version
1071             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, 'seed', ?, 'seed',
1072             1)
1073         """,
1074         (
1075             item_id,
1076             f"ITEM{index:03d}",
1077             f"物料{index:03d}",
1078             "MATERIAL" if index % 3 else "SERVICE",
1079             "EA",
1080             float(20 + index),
1081             float(50 + index * 3),
1082             STATUS_ACTIVE,
1083             now,
1084             now,
1085         ),
1086     )
1087 // empty line
1088 projects = []
1089 for index in range(1, 11):
1090     project_id = generate_id("proj")
1091     projects.append(project_id)
1092     connection.execute(
1093         """
1094         INSERT INTO projects(
1095             id, code, name, owner_org_id, start_date, end_date,
1096             status,
1097             created_at, created_by, updated_at, updated_by,
1098             version
1099             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, 'seed', ?, 'seed', 1)
1100         """,

```

```

1101         (
1102             project_id,
1103             f"PRJ{index:03d}",
1104             f"项目{index:03d}",
1105             org_north if index <= 5 else org_south,
1106             f"2026-0{(index % 9) + 1}-01",
1107             f"2026-12-{(index % 27) + 1:02d}",
1108             STATUS_ACTIVE,
1109             now,
1110             now,
1111         ),
1112     )
1113 // empty line
1114     warehouses = []
1115     warehouse_orgs = [org_north_wh, org_south_wh, org_north_wh,
1116 org_south_wh, org_hq]
1117     for index in range(1, 6):
1118         warehouse_id = generate_id("wh")
1119         warehouses.append(warehouse_id)
1120         connection.execute(
1121             """
1122             INSERT INTO warehouses(
1123                 id, code, name, org_unit_id, location, status,
1124                 created_at, created_by, updated_at, updated_by,
1125 version
1126             ) VALUES (?, ?, ?, ?, ?, ?, ?, 'seed', ?, 'seed', 1)
1127             """,
1128             (
1129                 warehouse_id,
1130                 f"WH{index:03d}",
1131                 f"仓库{index:03d}",
1132                 warehouse_orgs[index - 1],
1133                 f"城市{index:02d}仓",
1134                 STATUS_ACTIVE,
1135                 now,
1136                 now,
1137             ),
1138         )
1139 // empty line
1140     for month in range(1, 13):
1141         budget_id = generate_id("budget")
1142         budget_key = f"2026-{month:02d}-{org_north_proc}-
1143 {cost_centers[month % len(cost_centers)]}"
1144         connection.execute(
1145             """
1146             INSERT INTO budgets(
1147                 id, budget_year, budget_month, org_unit_id,
1148 project_id, cost_center_id,
1149                 budget_key, budget_amount, used_amount,
1150                 warning_amount, control_mode, status,

```

```

1151         created_at, created_by, updated_at, updated_by,
1152 version
1153     ) VALUES (?, 2026, ?, ?, ?, ?, ?, ?, ?, ?, 'HARD', ?, ?,
1154 'seed', ?, 'seed', 1)
1155     """,
1156     (
1157         budget_id,
1158         month,
1159         org_north_proc,
1160         projects[month % len(projects)],
1161         cost_centers[month % len(cost_centers)],
1162         budget_key,
1163         float(100000 + month * 5000),
1164         float(month * 1200),
1165         float(80000),
1166         STATUS_ACTIVE,
1167         now,
1168         now,
1169     ),
1170 )
1171 // empty line
1172     workflow_templates = [
1173         (
1174             BIZ_PROCUREMENT_REQUEST,
1175             "采购申请标准流程",
1176             {
1177                 "nodes": [
1178                     {"code": "DIRECTOR", "name": "运营总监审批",
1179 "assigneeRole": ROLE_OPERATIONS_DIRECTOR, "timeoutHours": 24},
1180                     {"code": "FINANCE", "name": "财务审批",
1181 "assigneeRole": ROLE_FINANCE, "timeoutHours": 24},
1182                 ],
1183                 "copyRoles": [ROLE_AUDITOR],
1184             },
1185         ),
1186         (
1187             BIZ_CONTRACT,
1188             "合同标准流程",
1189             {
1190                 "nodes": [
1191                     {"code": "DIRECTOR", "name": "运营总监审批",
1192 "assigneeRole": ROLE_OPERATIONS_DIRECTOR, "timeoutHours": 24},
1193                     {"code": "FINANCE", "name": "财务审批",
1194 "assigneeRole": ROLE_FINANCE, "timeoutHours": 24},
1195                 ],
1196                 "copyRoles": [ROLE_AUDITOR],
1197             },
1198         ),
1199         (
1200             BIZ_EXPENSE,

```

```

1201         "报销标准流程",
1202         {
1203             "nodes": [
1204                 {"code": "DIRECTOR", "name": "直属负责人审批",
1205 "assigneeRole": ROLE_OPERATIONS_DIRECTOR, "timeoutHours": 24},
1206                 {"code": "FINANCE", "name": "财务审批",
1207 "assigneeRole": ROLE_FINANCE, "timeoutHours": 24},
1208             ],
1209             "copyRoles": [ROLE_AUDITOR],
1210         },
1211     ),
1212 ]
1213 for biz_type, name, definition in workflow_templates:
1214     connection.execute(
1215         """
1216         INSERT INTO workflow_templates(
1217             id, biz_type, name, definition_json, enabled,
1218 status,
1219             created_at, created_by, updated_at, updated_by,
1220 version
1221             ) VALUES (?, ?, ?, ?, 1, ?, ?, 'seed', ?, 'seed', 1)
1222         """,
1223         (generate_id("wft"), biz_type, name, json.dumps
1224 (definition), STATUS_ACTIVE, now, now),
1225     )
1226 // empty line
1227     sales_orders = []
1228     for index in range(1, 11):
1229         order_id = generate_id("so")
1230         sales_orders.append(order_id)
1231         order_amount = float(10000 + index * 2500)
1232         items_json = json.dumps(
1233             [
1234                 {"itemId": items[index % len(items)], "qty": 5 +
1235 index, "unitPrice": 200 + index * 10},
1236                 {"itemId": items[(index + 1) % len(items)],
1237 "qty": 2 + index, "unitPrice": 180 + index * 12},
1238             ]
1239         )
1240         connection.execute(
1241             """
1242             INSERT INTO sales_orders(
1243                 id, order_no, customer_id, amount_total, currency,
1244 delivery_date, remark, items_json,
1245                 workflow_instance_id, status, created_at,
1246 created_by, updated_at, updated_by, version
1247                 ) VALUES (?, ?, ?, ?, 'CNY', ?, ?, ?, NULL, ?, ?, ?, ?,
1248 ?, 1)
1249             """,
1250             (

```

```

1251         order_id,
1252         generate_document_no("SO"),
1253         customers[index % len(customers)],
1254         order_amount,
1255         f"2026-06-{index:02d}",
1256         f"样例销售订单 {index}",
1257         items_json,
1258         "DRAFT" if index % 3 else STATUS_APPROVED,
1259         now,
1260         users["sales"],
1261         now,
1262         users["sales"],
1263     ),
1264 )
1265 // empty line
1266     contracts = []
1267     for index in range(1, 11):
1268         contract_id = generate_id("ct")
1269         contracts.append(contract_id)
1270         status = STATUS_EFFECTIVE if index <= 6 else
1271 (STATUS_PENDING if index <= 8 else STATUS_COMPLETED)
1272         effective_date = f"2026-05-{index:02d}" if status !=
1273 STATUS_PENDING else None
1274         expire_date = (datetime(2026, 12, 1) + timedelta
1275 (days=index * 5)).date().isoformat()
1276         receipt_progress = float(index * 10 if index <= 6 else 0)
1277         connection.execute(
1278             """
1279             INSERT INTO contracts(
1280                 id, contract_no, sales_order_id, amount_total,
1281 version_no, effective_date, expire_date,
1282                 receipt_progress, attachments_json,
1283 workflow_instance_id, status,
1284                 created_at, created_by, updated_at, updated_by,
1285 version
1286             ) VALUES (?, ?, ?, ?, 1, ?, ?, ?, ?, NULL, ?, ?, ?, ?,
1287 ?, 1)
1288             """
1289             (
1290                 contract_id,
1291                 generate_document_no("CT"),
1292                 sales_orders[index - 1],
1293                 float(15000 + index * 3000),
1294                 effective_date,
1295                 expire_date,
1296                 receipt_progress,
1297                 json.dumps([{"name": f"合同附件{index}.pdf"}]),
1298                 status,
1299                 now,
1300                 users["sales"],

```

```

1301         now,
1302         users["sales"],
1303     ),
1304 )
1305 // empty line
1306     procurement_requests = []
1307     purchase_orders = []
1308     for index in range(1, 11):
1309         request_id = generate_id("pr")
1310         procurement_requests.append(request_id)
1311         request_status = STATUS_APPROVED if index <=
1312 5 else STATUS_PENDING
1313         budget_key = f"2026-{{{(index - 1) % 12} + 1:02d}}-
1314 {org_north_proc}-{cost_centers[index % len(cost_centers)]}"
1315         request_items = [
1316             {"itemId": items[index % len(items)], "qty": 5 + index,
1317 "unitPrice": 120 + index * 8},
1318             {"itemId": items[(index + 2) % len(items)], "qty": 3 +
1319 index, "unitPrice": 80 + index * 7},
1320         ]
1321         amount_total = sum(item["qty"] * item
1322 ["unitPrice"] for item in request_items)
1323         connection.execute(
1324             """
1325             INSERT INTO procurement_requests(
1326                 id, request_no, request_org_id, cost_center_id,
1327 project_id, budget_key, amount_total, need_date,
1328                 items_json, budget_check_result,
1329 workflow_instance_id, status,
1330                 created_at, created_by, updated_at, updated_by,
1331 version
1332             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, NULL, ?, ?, ?,
1333 ?, ?, 1)
1334             """,
1335             (
1336                 request_id,
1337                 generate_document_no("PR"),
1338                 org_north_proc,
1339                 cost_centers[index % len(cost_centers)],
1340                 projects[index % len(projects)],
1341                 budget_key,
1342                 amount_total,
1343                 f"2026-07-{index:02d}",
1344                 json.dumps(request_items),
1345                 json.dumps({"allowed": True,
1346 "requestedAmount": amount_total}),
1347                 request_status,
1348                 now,
1349                 users["procurement"],
1350                 now,

```

```

1351         users["procurement"],
1352     ),
1353 )
1354 po_id = generate_id("po")
1355 purchase_orders.append(po_id)
1356 po_status = STATUS_COMPLETED if index <= 3 else
1357 (STATUS_PARTIAL if index <= 6 else STATUS_OPEN)
1358 received_amount = amount_total if po_status ==
1359 STATUS_COMPLETED else amount_total * 0.5 if po_status ==
1360 STATUS_PARTIAL else 0
1361 execution_rate = 100 if po_status ==
1362 STATUS_COMPLETED else 50 if po_status == STATUS_PARTIAL else 0
1363 connection.execute(
1364     """
1365     INSERT INTO purchase_orders(
1366         id, po_no, request_id, supplier_id, amount_total,
1367         expected_receipt_date,
1368         items_json, received_amount, execution_rate, status,
1369         created_at, created_by, updated_at, updated_by,
1370         version
1371     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, 1)
1372     """,
1373     (
1374         po_id,
1375         generate_document_no("PO"),
1376         request_id,
1377         suppliers[index % len(suppliers)],
1378         amount_total,
1379         f"2026-08-{{index:02d}}",
1380         json.dumps(request_items),
1381         received_amount,
1382         execution_rate,
1383         po_status,
1384         now,
1385         users["procurement"],
1386         now,
1387         users["procurement"],
1388     ),
1389 )
1390 // empty line
1391 for index in range(1, 11):
1392     stock_balance_id = generate_id("stock")
1393     qty_on_hand = float(40 + index * 6)
1394     connection.execute(
1395         """
1396         INSERT INTO stock_balances(
1397             id, warehouse_id, item_id, qty_on_hand,
1398             qty_reserved, status,
1399             created_at, created_by, updated_at, updated_by,
1400             version

```

```

1401         ) VALUES (?, ?, ?, ?, 0, ?, ?, 'seed', ?, 'seed', 1)
1402         """
1403     (
1404         stock_balance_id,
1405         warehouses[index % len(warehouses)],
1406         items[index % len(items)],
1407         qty_on_hand,
1408         STATUS_ACTIVE,
1409         now,
1410         now,
1411     ),
1412 )
1413 connection.execute(
1414     """
1415     INSERT INTO inventory_txns(
1416         id, txn_no, warehouse_id, item_id, txn_type, qty,
1417 biz_ref_type, biz_ref_id,
1418         remark, status, created_at, created_by, updated_at,
1419 updated_by, version
1420     ) VALUES (?, ?, ?, ?, 'RECEIPT', ?, 'SEED', ?, ?, ?, ?,
1421 'seed', ?, 'seed', 1)
1422     """
1423     (
1424         generate_id("txn"),
1425         generate_document_no("IT"),
1426         warehouses[index % len(warehouses)],
1427         items[index % len(items)],
1428         qty_on_hand,
1429         purchase_orders[index - 1],
1430         "期初库存导入",
1431         STATUS_ACTIVE,
1432         now,
1433         now,
1434     ),
1435 )
1436 // empty line
1437 expense_claims = []
1438 for index in range(1, 11):
1439     claim_id = generate_id("claim")
1440     expense_claims.append(claim_id)
1441     status = STATUS_APPROVED if index <= 4 else STATUS_PENDING
1442     amount_total = float(1500 + index * 260)
1443     budget_key = f"2026-{{{(index - 1) % 12} + 1:02d}}-
1444 {org_north_proc}-{cost_centers[index % len(cost_centers)]}"
1445     connection.execute(
1446         """
1447     INSERT INTO expense_claims(
1448         id, claim_no, claim_user_id, cost_center_id,
1449 project_id, amount_total, expense_type,
1450         attachments_json, budget_key, budget_check_result,

```

```

1451 workflow_instance_id, status,
1452         created_at, created_by, updated_at, updated_by,
1453 version
1454 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, NULL, ?, ?, ?,
1455 ?, ?, 1)
1456     """
1457     (
1458         claim_id,
1459         generate_document_no("EC"),
1460         users["sales"],
1461         cost_centers[index % len(cost_centers)],
1462         projects[index % len(projects)],
1463         amount_total,
1464         "TRAVEL" if index % 2 else "MARKETING",
1465         json_dumps([{"name": f"报销附件{index}.pdf"}]),
1466         budget_key,
1467         json_dumps({"allowed": True,
1468 "requestedAmount": amount_total}),
1469         status,
1470         now,
1471         users["sales"],
1472         now,
1473         users["sales"],
1474     ),
1475 )
1476 // empty line
1477     for index in range(1, 6):
1478         connection.execute(
1479             """
1480             INSERT INTO payment_requests(
1481                 id, payment_no, source_type, source_id, payee_name,
1482 amount_total, planned_date, status,
1483                 created_at, created_by, updated_at, updated_by,
1484 version
1485             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, 'seed', ?, 'seed',
1486 1)
1487             """
1488             (
1489                 generate_id("pay"),
1490                 generate_document_no("PAY"),
1491                 "EXPENSE_CLAIM",
1492                 expense_claims[index - 1],
1493                 f"收款方{index:02d}",
1494                 float(1000 + index * 200),
1495                 f"2026-09-{index:02d}",
1496                 STATUS_OPEN,
1497                 now,
1498                 now,
1499             ),
1500         )

```

```

1501 // empty line
1502     for index in range(1, 6):
1503         receipt_id = generate_id("receipt")
1504         amount = float(2000 + index * 800)
1505         connection.execute(
1506             """
1507             INSERT INTO receipt_records(
1508                 id, receipt_no, contract_id, amount, receipt_date,
1509 method, status,
1510                 created_at, created_by, updated_at, updated_by,
1511 version
1512             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'seed', ?, 'seed', 1)
1513             """,
1514             (
1515                 receipt_id,
1516                 generate_document_no("REC"),
1517                 contracts[index - 1],
1518                 amount,
1519                 f"2026-10-{index:02d}",
1520                 "BANK_TRANSFER",
1521                 STATUS_ACTIVE,
1522                 now,
1523                 now,
1524             ),
1525         )
1526 // empty line
1527     open_workflows = [
1528         (
1529             BIZ_PROCUREMENT_REQUEST,
1530             procurement_requests[7],
1531             "采购申请待审批",
1532             users["director"],
1533             users["procurement"],
1534             {"requestOrgId": org_north_proc, "amountTotal": 48000},
1535         ),
1536         (
1537             BIZ_EXPENSE,
1538             expense_claims[8],
1539             "费用报销待审批",
1540             users["director"],
1541             users["sales"],
1542             {"amountTotal": 3680},
1543         ),
1544         (
1545             BIZ_CONTRACT,
1546             contracts[7],
1547             "合同待审批",
1548             users["director"],
1549             users["sales"],
1550             {"amountTotal": 42000},

```

```

1551         ),
1552     ]
1553     for biz_type, biz_id, title, assignee_user_id,
1554     initiator_user_id, payload in open_workflows:
1555         instance_id = generate_id("wf")
1556         task_id = generate_id("task")
1557         connection.execute(
1558             """
1559             INSERT INTO workflow_instances(
1560                 id, biz_type, biz_id, title, template_id,
1561                 initiator_user_id,
1562                 current_node, result, payload_json, status,
1563                 created_at, created_by, updated_at, updated_by,
1564                 version
1565             ) VALUES (?, ?, ?, ?, ?, ?, 'DIRECTOR', NULL, ?,
1566             'IN_PROGRESS', ?, ?, ?, ?, 1)
1567             """
1568             (
1569                 instance_id,
1570                 biz_type,
1571                 biz_id,
1572                 title,
1573                 generate_id("wft_link"),
1574                 initiator_user_id,
1575                 json.dumps(payload),
1576                 now,
1577                 initiator_user_id,
1578                 now,
1579                 initiator_user_id,
1580             ),
1581         )
1582         connection.execute(
1583             """
1584             INSERT INTO approval_tasks(
1585                 id, instance_id, biz_type, biz_id, node_code,
1586                 node_name, assignee_user_id,
1587                 action, comment, due_at, status, created_at,
1588                 created_by, updated_at, updated_by, version
1589             ) VALUES (?, ?, ?, ?, 'DIRECTOR', '运营总监审批', ?, NULL,
1590             NULL, ?, ?, ?, ?, ?, ?, 1)
1591             """
1592             (
1593                 task_id,
1594                 instance_id,
1595                 biz_type,
1596                 biz_id,
1597                 assignee_user_id,
1598                 (datetime.utcnow() + timedelta(hours=24)).isoformat
1599             () + "Z",
1600                 WORKFLOW_STATUS_OPEN,

```

```

1601         now,
1602         initiator_user_id,
1603         now,
1604         initiator_user_id,
1605     ),
1606 )
1607 // empty line
1608     done_workflows = [
1609         (BIZ_PROCUREMENT_REQUEST, procurement_requests[1], users
1610 ["finance"], users["procurement"]),
1611         (BIZ_EXPENSE, expense_claims[1], users["finance"], users
1612 ["sales"]),
1613         (BIZ_CONTRACT, contracts[1], users["finance"], users
1614 ["sales"]),
1615     ]
1616     for biz_type, biz_id, assignee_user_id,
1617 initiator_user_id in done_workflows:
1618         instance_id = generate_id("wf")
1619         task_id = generate_id("task")
1620         connection.execute(
1621             """
1622             INSERT INTO workflow_instances(
1623                 id, biz_type, biz_id, title, template_id,
1624 initiator_user_id,
1625                 current_node, result, payload_json, status,
1626                 created_at, created_by, updated_at, updated_by,
1627 version
1628             ) VALUES (?, ?, ?, ?, ?, ?, NULL, ?, '{}', 'COMPLETED',
1629 ?, ?, ?, ?, 1)
1630             """
1631             (
1632                 instance_id,
1633                 biz_type,
1634                 biz_id,
1635                 f"{biz_type} 已审批",
1636                 generate_id("wft_link"),
1637                 initiator_user_id,
1638             align-items: end;
1639         }
1640 // empty line
1641 .page-header h2,
1642 .login-card h1 {
1643     margin: 0;
1644     font-family: var(--font-heading);
1645     font-size: 34px;
1646     line-height: 1.15;
1647 }
1648 // empty line
1649 .eyebrow,
1650 .hero-kicker {

```

```
1651     letter-spacing: 0.18em;
1652     text-transform: uppercase;
1653     color: var(--accent);
1654     font-size: 12px;
1655     margin: 0 0 10px;
1656 }
1657 // empty line
1658 .metric-grid {
1659     display: grid;
1660     grid-template-columns: repeat(3, minmax(0, 1fr));
1661     gap: 16px;
1662 }
1663 // empty line
1664 .metric-grid.compact {
1665     grid-template-columns: repeat(5, minmax(0, 1fr));
1666 }
1667 // empty line
1668 .metric-card,
1669 .panel-card {
1670     border-radius: 22px;
1671     border: 1px solid rgba(44, 62, 80, 0.08);
1672     background: var(--surface);
1673     backdrop-filter: blur(18px);
1674     box-shadow: var(--shadow);
1675 }
1676 // empty line
1677 .metric-card {
1678     padding: 18px;
1679 }
1680 // empty line
1681 .metric-title,
1682 .muted {
1683     color: var(--muted);
1684 }
1685 // empty line
1686 .metric-value {
1687     display: block;
1688     margin: 10px 0 8px;
1689     font-size: 30px;
1690 }
1691 // empty line
1692 .metric-hint {
1693     font-size: 13px;
1694     color: var(--muted);
1695 }
1696 // empty line
1697 .panel-grid {
1698     display: grid;
1699     grid-template-columns: repeat(2, minmax(0, 1fr));
1700     gap: 18px;
```

```
1701     }
1702     // empty line
1703     .panel-card {
1704         padding: 18px;
1705     }
1706     // empty line
1707     .panel-head {
1708         display: flex;
1709         align-items: center;
1710         justify-content: space-between;
1711         margin-bottom: 14px;
1712     }
1713     // empty line
1714     .panel-head h3 {
1715         margin: 0;
1716         font-size: 18px;
1717     }
1718     // empty line
1719     .bar-stack {
1720         display: grid;
1721         gap: 14px;
1722     }
1723     // empty line
1724     .bar-row {
1725         display: grid;
1726         grid-template-columns: 58px 1fr 88px;
1727         align-items: center;
1728         gap: 10px;
1729     }
1730     // empty line
1731     .bar-row div {
1732         height: 10px;
1733         border-radius: 999px;
1734         background: rgba(15, 118, 110, 0.09);
1735         overflow: hidden;
1736     }
1737     // empty line
1738     .bar-row i {
1739         display: block;
1740         height: 100%;
1741         border-radius: 999px;
1742         background: linear-gradient(90deg, var(--brand), var(--accent));
1743     }
1744     // empty line
1745     .table-shell {
1746         overflow: auto;
1747     }
1748     // empty line
1749     table {
1750         width: 100%;
```

```
1751     border-collapse: collapse;
1752 }
1753 // empty line
1754 th,
1755 td {
1756     padding: 12px 10px;
1757     text-align: left;
1758     border-bottom: 1px solid rgba(44, 62, 80, 0.08);
1759     font-size: 14px;
1760 }
1761 // empty line
1762 th {
1763     color: var(--muted);
1764     font-weight: 600;
1765 }
1766 // empty line
1767 .grid-form {
1768     display: grid;
1769     gap: 12px;
1770 }
1771 // empty line
1772 .grid-form label {
1773     display: grid;
1774     gap: 8px;
1775 }
1776 // empty line
1777 .grid-form.inline-3 {
1778     grid-template-columns: repeat(3, minmax(0, 1fr)) auto;
1779     align-items: end;
1780 }
1781 // empty line
1782 .grid-form.inline-4 {
1783     grid-template-columns: repeat(4, minmax(0, 1fr)) auto;
1784     align-items: end;
1785 }
1786 // empty line
1787 input {
1788     width: 100%;
1789     padding: 12px 14px;
1790     border-radius: 14px;
1791     border: 1px solid rgba(44, 62, 80, 0.14);
1792     background: rgba(255, 255, 255, 0.85);
1793 }
1794 // empty line
1795 .primary-button,
1796 .secondary-button,
1797 .mini-button {
1798     border: 0;
1799     cursor: pointer;
1800     border-radius: 14px;
```

```
1801     padding: 12px 16px;
1802     transition: 160ms ease;
1803 }
1804 // empty line
1805 .primary-button {
1806     background: linear-gradient(135deg, var(--brand), #15867d);
1807     color: #fff;
1808 }
1809 // empty line
1810 .secondary-button,
1811 .mini-button {
1812     background: rgba(15, 118, 110, 0.1);
1813     color: var(--brand);
1814 }
1815 // empty line
1816 .mini-button {
1817     padding: 8px 12px;
1818     font-size: 13px;
1819 }
1820 // empty line
1821 .mini-button.ghost {
1822     background: rgba(166, 61, 64, 0.1);
1823     color: var(--danger);
1824 }
1825 // empty line
1826 .status-pill {
1827     display: inline-flex;
1828     align-items: center;
1829     padding: 6px 10px;
1830     border-radius: 999px;
1831     background: rgba(15, 118, 110, 0.1);
1832     color: var(--brand);
1833     font-size: 12px;
1834 }
1835 // empty line
1836 .task-list {
1837     display: grid;
1838     gap: 12px;
1839 }
1840 // empty line
1841 .task-card {
1842     border-radius: 18px;
1843     border: 1px solid rgba(44, 62, 80, 0.08);
1844     padding: 14px;
1845     background: rgba(255, 255, 255, 0.72);
1846     display: grid;
1847     gap: 6px;
1848 }
1849 // empty line
1850 .task-card.done {
```

```
1851     background: rgba(230, 244, 242, 0.78);
1852 }
1853 // empty line
1854 .task-actions {
1855     display: flex;
1856     gap: 8px;
1857     margin-top: 6px;
1858 }
1859 // empty line
1860 .alert-list {
1861     list-style: none;
1862     padding: 0;
1863     margin: 0;
1864     display: grid;
1865     gap: 10px;
1866 }
1867 // empty line
1868 .alert-item {
1869     display: grid;
1870     grid-template-columns: 110px 1fr 110px;
1871     gap: 12px;
1872     align-items: center;
1873     padding: 12px 0;
1874     border-bottom: 1px solid rgba(44, 62, 80, 0.08);
1875 }
1876 // empty line
1877 .action-form {
1878     display: grid;
1879     gap: 10px;
1880 }
1881 // empty line
1882 .login-shell {
1883     min-height: 100vh;
1884     display: grid;
1885     grid-template-columns: minmax(0, 560px) minmax(0, 1fr);
1886     padding: 40px;
1887     gap: 24px;
1888 }
1889 // empty line
1890 .login-card,
1891 .login-panel {
1892     border-radius: 28px;
1893     padding: 34px;
1894     box-shadow: var(--shadow);
1895     border: 1px solid rgba(44, 62, 80, 0.08);
1896 }
1897 // empty line
1898 .login-card {
1899     background: linear-gradient(180deg, rgba(255, 253, 249, 0.96), rgba
1900 (250, 255, 254, 0.86));
```

```
1901     }
1902     // empty line
1903     .login-panel {
1904         background:
1905             linear-gradient(135deg, rgba(15, 118, 110, 0.12), rgba(186, 111, 45,
1906             0.12)),
1907             rgba(255, 255, 255, 0.72);
1908         display: grid;
1909         align-content: center;
1910     }
1911     // empty line
1912     .tip-strip {
1913         display: flex;
1914         flex-wrap: wrap;
1915         gap: 10px;
1916         margin-top: 16px;
1917     }
1918     // empty line
1919     .tip-strip span,
1920     .panel-card ul li {
1921         color: var(--muted);
1922         font-size: 14px;
1923     }
1924     // empty line
1925     .panel-card ul {
1926         margin: 12px 0 0;
1927         padding-left: 18px;
1928     }
1929     // empty line
1930     @media (max-width: 1280px) {
1931         .app-shell {
1932             grid-template-columns: 240px 1fr;
1933         }
1934     // empty line
1935     .metric-grid.compact {
1936         grid-template-columns: repeat(3, minmax(0, 1fr));
1937     }
1938     // empty line
1939     .grid-form.inline-4 {
1940         grid-template-columns: repeat(2, minmax(0, 1fr));
1941     }
1942     }
1943     // empty line
1944     @media (max-width: 1024px) {
1945         .app-shell,
1946         .login-shell,
1947         .panel-grid,
1948         .metric-grid {
1949             grid-template-columns: 1fr;
1950         }
```

```

1951 // empty line
1952 .sidebar {
1953   border-right: 0;
1954   border-bottom: 1px solid var(--line);
1955 }
1956 // empty line
1957 .grid-form.inline-3,
1958 .grid-form.inline-4 {
1959   grid-template-columns: 1fr;
1960 }
1961 }
1962 // section delimiter
1963 apps/web/src/views/login.js
1964 import { api } from "../api.js";
1965 import { setFlash, setSession } from "../state.js";
1966 import { t } from "../i18n.js";
1967 // empty line
1968 export function renderLogin(root, app) {
1969   root.innerHTML = `
1970     <section class="login-shell">
1971       <div class="login-card">
1972         <div class="hero-kicker">B2B OPERATIONS CONTROL</div>
1973         <h1>${t("loginTitle")}</h1>
1974         <p class="muted">${t("loginSubtitle")}</p>
1975         <form id="login-form" class="grid-form">
1976           <label>
1977             <span>${t("loginUser")}</span>
1978             <input name="username" value="admin" autocomplete="username"
1979 //
1980             </label>
1981             <label>
1982               <span>${t("loginPassword")}</span>
1983               <input type="password" name="password" value="admin123" auto
1984 complete="current-password" />
1985             </label>
1986             <button class="primary-button" type="submit">${t
1987 ("loginAction")}</button>
1988           </form>
1989           <div class="tip-strip">
1990             <span>样例账号: admin / admin123</span>
1991             <span>审批账号: director / admin123</span>
1992             <span>财务账号: fin01 / admin123</span>
1993           </div>
1994         </div>
1995         <div class="login-panel">
1996           <div class="panel-card">
1997             <strong>首期覆盖模块</strong>
1998             <ul>
1999               <li>统一登录、角色权限、组织与成本中心</li>
2000               <li>采购申请、采购订单、收货入库、库存余额</li>

```

```

2001         <li>销售订单、合同审批、回款登记</li>
2002         <li>预算控制、费用报销、付款申请、审计通知</li>
2003     </ul>
2004 </div>
2005 </div>
2006 </section>
2007 `;
2008 root.querySelector("#login-form").addEventListener("submit", async
2009 (event) => {
2010     event.preventDefault();
2011     const formData = new FormData(event.currentTarget);
2012     try {
2013         const payload = await api.login({
2014             username: formData.get("username"),
2015             password: formData.get("password"),
2016         });
2017         setSession(payload);
2018         setFlash("success", "登录成功");
2019         await app.bootstrapUser();
2020         app.navigate("dashboard");
2021     } catch (error) {
2022         app.showError(error);
2023     }
2024 });
2025 }
2026 // section delimiter
2027 apps/web/src/views/dashboard.js
2028 import { api } from "../api.js";
2029 import { t } from "../i18n.js";
2030 // empty line
2031 function metricCard(title, value, hint) {
2032     return `
2033         <article class="metric-card">
2034             <span class="metric-title">${title}</span>
2035             <strong class="metric-value">${value}</strong>
2036             <span class="metric-hint">${hint}</span>
2037         </article>
2038     `;
2039 }
2040 // empty line
2041 function alertItem(item) {
2042     return `
2043         <li class="alert-item">
2044             <strong>${item.type}</strong>
2045             <span>${item.label}</span>
2046             <em>${item.value}</em>
2047         </li>
2048     `;
2049 }
2050 // empty line

```

```

2051 function kpiBar(label, amount, maxReference) {
2052   const percent = Math.min((amount / maxReference) * 100, 100);
2053   return `
2054     <div class="bar-row">
2055       <span>${label}</span>
2056       <div><i class="bar-fill" data-percent="${percent.toFixed
2057 (2)}"></i></div>
2058       <strong>${amount}</strong>
2059     </div>
2060 `;
2061 }
2062 // empty line
2063 export async function renderDashboard(root) {
2064   const dashboardData = await api.dashboard();
2065   root.innerHTML = `
2066     <section class="page-header">
2067       <div>
2068         <p class="eyebrow">OPERATIONS COMMAND</p>
2069         <h2>${t("dashboardTitle")}</h2>
2070       </div>
2071     </section>
2072     <section class="metric-grid">
2073       ${metricCard("销售订单数", dashboardData.salesKpi.orderCount, `合同 $
2074 {dashboardData.salesKpi.contractCount} 份`)}
2075       ${metricCard("采购申请数", dashboardData.procurementKpi.requestCount,
2076 `待审 ${dashboardData.procurementKpi.pendingRequestCount} 条`)}
2077       ${metricCard("库存 SKU", dashboardData.inventoryKpi.stockSkuCount,
2078 `库存总量 ${dashboardData.inventoryKpi.stockQty}`)}
2079       ${metricCard("报销单数", dashboardData.expenseKpi.claimCount, `回款金额 $
2080 {dashboardData.expenseKpi.receiptAmount}`)}
2081       ${metricCard("我的待办", dashboardData.todoKpi.todoCount, `已办 $
2082 {dashboardData.todoKpi.doneCount}`)}
2083       ${metricCard("未读通知", dashboardData.todoKpi.unreadNotifications,
2084 "站内信与预警")}
2085     </section>
2086     <section class="panel-grid">
2087       <article class="panel-card">
2088         <header class="panel-head"><h3>经营金额概览</h3></header>
2089         <div class="bar-stack">
2090           ${kpiBar("销售", dashboardData.salesKpi.orderAmount, 2000)}
2091           ${kpiBar("采购", dashboardData.procurementKpi.poAmount, 2000)}
2092           ${kpiBar("回款", dashboardData.expenseKpi.receiptAmount, 2000)}
2093         </div>
2094       </article>
2095       <article class="panel-card">
2096         <header class="panel-head"><h3>风险提示</h3></header>
2097         <ul class="alert-list">
2098           ${((dashboardData.alerts || []).map(alertItem).join("") ||
2099 `<li class="muted">${t("emptyState")}</li>`)}
2100         </ul>

```

```

2101         </article>
2102     </section>
2103     `;
2104 // empty line
2105     root.querySelectorAll(".bar-fill").forEach((element) => {
2106         element.style.width = `${element.dataset.percent || "0"}%`;
2107     });
2108 }
2109 // section delimiter
2110 apps/web/src/views/masterdata.js
2111 import { api } from "../api.js";
2112 // empty line
2113 function table(title, items, columns) {
2114     const head = columns.map((column) => `<th>${column.label}</th>`).join
2115     ("");
2116     const rows = items
2117         .map(
2118             (item) => `
2119                 <tr>
2120                     ${columns.map((column) => `<td>${item
2121 [column.key] ?? ""}</td>`).join("")}
2122                 </tr>
2123             `
2124         )
2125         .join("");
2126     return `
2127         <article class="panel-card">
2128             <header class="panel-head"><h3>${title}</h3></header>
2129             <div class="table-shell">
2130                 <table>
2131                     <thead><tr>${head}</tr></thead>
2132                     <tbody>${rows || `<tr><td colspan="$
2133 {columns.length}">暂无数据</td></tr>`}</tbody>
2134                 </table>
2135             </div>
2136         </article>
2137     `;
2138 }
2139 // empty line
2140 export async function renderMasterdata(root, app) {
2141     const [summary, customers, suppliers, items, projects, warehouses] =
2142     await Promise.all([
2143         api.summary(),
2144         api.customers(),
2145         api.suppliers(),
2146         api.items(),
2147         api.projects(),
2148         api.warehouses(),
2149     ]);
2150     root.innerHTML = `

```

```

2151     <section class="page-header">
2152         <div>
2153             <p class="eyebrow">MASTER DATA</p>
2154             <h2>主数据与组织基线</h2>
2155         </div>
2156     </section>
2157     <section class="metric-grid compact">
2158         <article class="metric-card"><span class="metric-title">客户</span><
2159     strong class="metric-value">${summary.customers}</strong></article>
2160         <article class="metric-card"><span class="metric-title">供应商</span><
2161     <strong class="metric-value">${summary.suppliers}</strong></article>
2162         <article class="metric-card"><span class="metric-title">物料</span><
2163     strong class="metric-value">${summary.items}</strong></article>
2164         <article class="metric-card"><span class="metric-title">项目</span><
2165     strong class="metric-value">${summary.projects}</strong></article>
2166         <article class="metric-card"><span class="metric-title">仓库</span><
2167     strong class="metric-value">${summary.warehouses}</strong></article>
2168     </section>
2169     <section class="action-form panel-card">
2170         <header class="panel-head"><h3>快速创建客户</h3></header>
2171         <form id="customer-form" class="grid-form inline-3">
2172             <input name="code" placeholder="客户编码, 如 CUST900" />
2173             <input name="name" placeholder="客户名称" />
2174             <input name="industry" placeholder="行业, 如 制造" />
2175             <button class="primary-button" type="submit">新增客户</button>
2176         </form>
2177     </section>
2178     <section class="panel-grid">
2179         ${table("客户", customers.items, [{ key: "code", label: "编码" },
2180     { key: "name", label: "名称" }, { key: "industry", label: "行业" }])}
2181         ${table("供应商", suppliers.items, [{ key: "code", label: "编码" },
2182     { key: "name", label: "名称" }, { key: "category", label: "类别" }])}
2183     </section>
2184     <section class="panel-grid">
2185         ${table("物料", items.items, [{ key: "code", label: "编码" },
2186     { key: "name", label: "名称" }, { key: "item_type", label: "类型" },
2187     { key: "safety_stock", label: "安全库存" }])}
2188         ${table("项目与仓库", [...projects.items.slice(0, 4),
2189     ...warehouses.items.slice(0, 4)], [{ key: "code", label: "编码" },
2190     { key: "name", label: "名称" }, { key: "status", label: "状态" }])}
2191     </section>
2192     `;
2193     root.querySelector("#customer-form").addEventListener("submit",
2194     async (event) => {
2195         event.preventDefault();
2196         const formData = new FormData(event.currentTarget);
2197         try {
2198             await api.createCustomer({
2199                 code: formData.get("code"),
2200                 name: formData.get("name"),

```

```

2201         industry: formData.get("industry"),
2202     });
2203     app.showSuccess("客户创建成功");
2204     await renderMasterdata(root, app);
2205     } catch (error) {
2206         app.showError(error);
2207     }
2208 });
2209 }
2210 // section delimiter
2211 apps/web/src/views/procurement.js
2212 import { api } from "../api.js";
2213 // empty line
2214 function requestRows(items) {
2215     return items
2216         .map(
2217             (item) => `
2218                 <tr>
2219                     <td>${item.request_no}</td>
2220                     <td>${item.amount_total}</td>
2221                     <td>${item.need_date}</td>
2222                     <td><span class="status-pill">${item.status}</span></td>
2223                 </tr>
2224             `
2225         )
2226         .join("");
2227 }
2228 // empty line
2229 function orderRows(items) {
2230     return items
2231         .map(
2232             (item) => `
2233                 <tr>
2234                     <td>${item.po_no}</td>
2235                     <td>${item.amount_total}</td>
2236                     <td>${item.execution_rate}%</td>
2237                     <td><span class="status-pill">${item.status}</span></td>
2238                 </tr>
2239             `
2240         )
2241         .join("");
2242 }
2243 // empty line
2244 export async function renderProcurement(root, app) {
2245     const [requests, orders, items, suppliers, warehouses, balances] =
2246     await Promise.all([
2247         api.procurementRequests(),
2248         api.purchaseOrders(),
2249         api.items(),
2250         api.suppliers(),

```

```

2251     api.warehouses(),
2252     api.stockBalances(),
2253   ]);
2254   const firstItem = items.items[0];
2255   const firstSupplier = suppliers.items[0];
2256   const firstWarehouse = warehouses.items[0];
2257   root.innerHTML = `
2258     <section class="page-header">
2259       <div>
2260         <p class="eyebrow">PROCUREMENT & INVENTORY</p>
2261         <h2>采购申请、采购订单与收货入库</h2>
2262       </div>
2263     </section>
2264     <section class="panel-card">
2265       <header class="panel-head"><h3>发起采购申请</h3></header>
2266       <form id="request-form" class="grid-form inline-4">
2267         <input name="requestOrgId" value="org_dummy_replaced" placeholder=
2268 r="申请组织 ID" />
2269         <input name="costCenterId" value="cc_dummy_replaced" placeholder
2270 ="成本中心 ID" />
2271         <input name="budgetKey" placeholder="预算键" />
2272         <input name="needDate" type="date" />
2273         <button class="primary-button" type="submit">提交申请</button>
2274       </form>
2275       <p class="muted">默认将使用样例物料 ${firstItem?.name || ""} 生成申请明细。</p>
2276     </section>
2277     <section class="panel-grid">
2278       <article class="panel-card">
2279         <header class="panel-head"><h3>采购申请</h3></header>
2280         <div class="table-shell">
2281           <table>
2282             <thead><tr><th>申请单号</th><th>金额</th><th>需求日期</th><th>状态</th><
2283 /tr></thead>
2284             <tbody>${requestRows(requests.items)}</tbody>
2285           </table>
2286         </div>
2287       </article>
2288       <article class="panel-card">
2289         <header class="panel-head"><h3>采购订单</h3></header>
2290         <div class="table-shell">
2291           <table>
2292             <thead><tr><th>订单号</th><th>金额</th><th>执行率</th><th>状态</th></tr></thead>
2293 r></thead>
2294             <tbody>${orderRows(orders.items)}</tbody>
2295           </table>
2296         </div>
2297       </article>
2298     </section>
2299     <section class="panel-grid">
2300       <article class="panel-card">

```

```

2301     <header class="panel-head"><h3>生成采购订单</h3></header>
2302     <form id="po-form" class="grid-form inline-3">
2303         <input name="requestId" placeholder="审批通过的采购申请 ID" />
2304         <input name="supplierId" value="{firstSupplier?.id ||
2305 ""}" placeholder="供应商 ID" />
2306         <input name="expectedReceiptDate" type="date" />
2307         <button class="secondary-button" type="submit">创建采购订单</button>
2308     </form>
2309 </article>
2310 <article class="panel-card">
2311     <header class="panel-head"><h3>收货入库</h3></header>
2312     <form id="receipt-form" class="grid-form inline-3">
2313         <input name="purchaseOrderId" placeholder="采购订单 ID" />
2314         <input name="warehouseId" value="{firstWarehouse?.id ||
2315 ""}" placeholder="仓库 ID" />
2316         <input name="qty" type="number" min="1" value="5" />
2317         <button class="secondary-button" type="submit">执行入库</button>
2318     </form>
2319 </article>
2320 </section>
2321 <section class="panel-card">
2322     <header class="panel-head"><h3>库存余额</h3></header>
2323     <div class="table-shell">
2324         <table>
2325             <thead><tr><th>库存记录 ID</th><th>仓库</th><th>物料</th><th>现存量</th><
2326 /tr></thead>
2327             <tbody>
2328                 ${balances.items
2329                     .map(
2330                     (item) => `
2331                     <tr>
2332                         <td>${item.id}</td>
2333                         <td>${item.warehouse_id}</td>
2334                         <td>${item.item_id}</td>
2335                         <td>${item.qty_on_hand}</td>
2336                     </tr>
2337                     `
2338                     )
2339                     .join("")}
2340             </tbody>
2341         </table>
2342     </div>
2343 </section>
2344 `;
2345 const presetOrg = requests.items[0]?.request_org_id || "";
2346 const presetCostCenter = requests.items[0]?.cost_center_id || "";
2347 root.querySelector("input[name='requestOrgId']").value = presetOrg;
2348 root.querySelector("input[name='costCenterId']").value =
2349 presetCostCenter;
2350 root.querySelector("input[name='budgetKey']").value = requests.items

```

```
2351 [0]?.budget_key || "";
2352 // empty line
2353 root.querySelector("#request-form").addEventListener("submit", async
2354 (event) => {
2355     event.preventDefault();
2356     const formData = new FormData(event.currentTarget);
2357     try {
2358         await api.createProcurementRequest({
2359             requestOrgId: formData.get("requestOrgId"),
2360             costCenterId: formData.get("costCenterId"),
2361             budgetKey: formData.get("budgetKey"),
2362             needDate: formData.get("needDate"),
2363             items: [{ itemId: firstItem.id, qty: 3,
2364 unitPrice: firstItem.unit_price || 120 }],
2365         });
2366         app.showSuccess("采购申请已提交并进入审批");
2367         await renderProcurement(root, app);
2368     } catch (error) {
2369         app.showError(error);
2370     }
2371 });
2372 // empty line
2373 root.querySelector("#po-form").addEventListener("submit", async
2374 (event) => {
2375     event.preventDefault();
2376     const formData = new FormData(event.currentTarget);
2377     try {
2378         await api.createPurchaseOrder({
2379             requestId: formData.get("requestId"),
2380             supplierId: formData.get("supplierId"),
2381             expectedReceiptDate: formData.get("expectedReceiptDate"),
2382         });
2383         app.showSuccess("采购订单已生成");
2384         await renderProcurement(root, app);
2385     } catch (error) {
2386         app.showError(error);
2387     }
2388 });
2389 // empty line
2390 root.querySelector("#receipt-form").addEventListener("submit", async
2391 (event) => {
2392     event.preventDefault();
2393     const formData = new FormData(event.currentTarget);
2394     try {
2395         await api.receiveInventory({
2396             purchaseOrderId: formData.get("purchaseOrderId"),
2397             warehouseId: formData.get("warehouseId"),
2398             items: [{ itemId: firstItem.id, qty: Number(formData.get
2399 ("qty")) }],
2400         });
```

```

2401     app.showSuccess("入库成功");
2402     await renderProcurement(root, app);
2403   } catch (error) {
2404     app.showError(error);
2405   }
2406   });
2407 }
2408 // section delimiter
2409 apps/web/src/views/sales.js
2410 import { api } from "../api.js";
2411 // empty line
2412 export async function renderSales(root, app) {
2413   const [orders, contracts, customers, items] = await Promise.all([
2414     api.salesOrders(),
2415     api.contracts(),
2416     api.customers(),
2417     api.items(),
2418   ]);
2419   const firstCustomer = customers.items[0];
2420   const firstItem = items.items[0];
2421   root.innerHTML = `
2422     <section class="page-header">
2423       <div>
2424         <p class="eyebrow">SALES & CONTRACT</p>
2425         <h2>销售订单与合同审批</h2>
2426       </div>
2427     </section>
2428     <section class="panel-grid">
2429       <article class="panel-card">
2430         <header class="panel-head"><h3>创建销售订单</h3></header>
2431         <form id="sales-form" class="grid-form inline-3">
2432           <input name="customerId" value="${firstCustomer?.id ||
2433 ""}" placeholder="客户 ID" />
2434           <input name="deliveryDate" type="date" />
2435           <input name="amountTotal" type="number" min="1" value="12000"
2436 />
2437           <button class="primary-button" type="submit">新建销售订单</button>
2438         </form>
2439       </article>
2440       <article class="panel-card">
2441         <header class="panel-head"><h3>创建合同</h3></header>
2442         <form id="contract-form" class="grid-form inline-3">
2443           <input name="salesOrderId" placeholder="销售订单 ID" />
2444           <input name="expireDate" type="date" />
2445           <input name="amountTotal" type="number" min="1" value="18000"
2446 />
2447           <button class="secondary-button" type="submit">发起合同审批</button>
2448         </form>
2449       </article>
2450     </section>

```

```
2451     <section class="panel-grid">
2452       <article class="panel-card">
2453         <header class="panel-head"><h3>销售订单</h3></header>
2454         <div class="table-shell">
2455           <table>
2456             <thead><tr><th>订单号</th><th>客户</th><th>金额</th><th>状态</th></tr>
2457           </thead>
2458           <tbody>
2459             ${orders.items
2460               .map(
2461                 (item) => `
2462                 <tr>
2463                   <td>${item.order_no}</td>
2464                   <td>${item.customer_id}</td>
2465                   <td>${item.amount_total}</td>
2466                   <td><span class="status-pill">${
2467 {item.status}</span></td>
2468                 `
2469               ,
2470             )
2471               .join("")}
2472             </tbody>
2473           </table>
2474         </div>
2475       </article>
2476       <article class="panel-card">
2477         <header class="panel-head"><h3>合同</h3></header>
2478         <div class="table-shell">
2479           <table>
2480             <thead><tr><th>合同号</th><th>金额</th><th>回款进度</th><th>状态</th></tr>
2481           </thead>
2482           <tbody>
2483             ${contracts.items
2484               .map(
2485                 (item) => `
2486                 <tr>
2487                   <td>${item.contract_no}</td>
2488                   <td>${item.amount_total}</td>
2489                   <td>${item.receipt_progress}%</td>
2490                   <td><span class="status-pill">${
2491 {item.status}</span></td>
2492                 `
2493               ,
2494             )
2495               .join("")}
2496             </tbody>
2497           </table>
2498         </div>
2499       </article>
2500     </section>
```

```
2501 `;
2502 root.querySelector("#sales-form").addEventListener("submit", async
2503 (event) => {
2504   event.preventDefault();
2505   const formData = new FormData(event.currentTarget);
2506   try {
2507     await api.createSalesOrder({
2508       customerId: formData.get("customerId"),
2509       deliveryDate: formData.get("deliveryDate"),
2510       amountTotal: Number(formData.get("amountTotal")),
2511       items: [{ itemId: firstItem.id, qty: 10,
2512 unitPrice: firstItem.unit_price || 200 }],
2513     });
2514     app.showSuccess("销售订单创建成功");
2515     await renderSales(root, app);
2516   } catch (error) {
2517     app.showError(error);
2518   }
2519 });
2520 root.querySelector("#contract-form").addEventListener("submit",
2521 async (event) => {
2522   event.preventDefault();
2523   const formData = new FormData(event.currentTarget);
2524   try {
2525     await api.createContract({
2526       salesOrderId: formData.get("salesOrderId"),
2527       expireDate: formData.get("expireDate"),
2528       amountTotal: Number(formData.get("amountTotal")),
2529       attachments: [{ name: "样例合同.pdf" }],
2530     });
2531     app.showSuccess("合同已提交审批");
2532     await renderSales(root, app);
2533   } catch (error) {
2534     app.showError(error);
2535   }
2536 });
2537 }
2538 // section delimiter
2539 apps/web/src/views/finance.js
2540 import { api } from "../api.js";
2541 // empty line
2542 export async function renderFinance(root, app) {
2543   const [budgets, expenses, payments, receipts] = await Promise.all([
2544     api.budgets(),
2545     api.expenses(),
2546     api.payments(),
2547     api.receipts(),
2548   ]);
2549   const firstBudget = budgets.items[0];
2550   const firstExpense = expenses.items[0];
```

```
2551 const firstContractReceipt = receipts.items[0];
2552 root.innerHTML = `
2553   <section class="page-header">
2554     <div>
2555       <p class="eyebrow">FINANCE CONTROL</p>
2556       <h2>预算、报销、付款与回款</h2>
2557     </div>
2558   </section>
2559   <section class="panel-grid">
2560     <article class="panel-card">
2561       <header class="panel-head"><h3>新增预算</h3></header>
2562       <form id="budget-form" class="grid-form inline-4">
2563         <input name="budgetYear" type="number" value="2026" />
2564         <input name="budgetMonth" type="number" value="11" />
2565         <input name="budgetAmount" type="number" value="150000" />
2566         <input name="budgetKey" placeholder="预算键" />
2567         <button class="primary-button" type="submit">新增预算</button>
2568       </form>
2569     </article>
2570     <article class="panel-card">
2571       <header class="panel-head"><h3>发起费用报销</h3></header>
2572       <form id="expense-form" class="grid-form inline-4">
2573         <input name="claimUserId" value="${app.state.user?.id ||
2574 ""}" />
2575         <input name="costCenterId" value="${
2576 {firstBudget?.cost_center_id || ""}" />
2577         <input name="budgetKey" value="${firstBudget?.budget_key ||
2578 ""}" />
2579         <input name="amountTotal" type="number" value="3200" />
2580         <button class="secondary-button" type="submit">提交报销</button>
2581       </form>
2582     </article>
2583   </section>
2584   <section class="panel-grid">
2585     <article class="panel-card">
2586       <header class="panel-head"><h3>创建付款申请</h3></header>
2587       <form id="payment-form" class="grid-form inline-4">
2588         <input name="sourceType" value="EXPENSE_CLAIM" />
2589         <input name="sourceId" value="${firstExpense?.id || ""}" />
2590         <input name="payeeName" placeholder="收款方" value="广州分公司" />
2591         <input name="amountTotal" type="number" value="3200" />
2592         <button class="secondary-button" type="submit">发起付款</button>
2593       </form>
2594     </article>
2595     <article class="panel-card">
2596       <header class="panel-head"><h3>登记回款</h3></header>
2597       <form id="receipt-form" class="grid-form inline-4">
2598         <input name="contractId" value="${receipts.items
2599 [0]?.contract_id || ""}" />
2600         <input name="amount" type="number" value="5000" />
```

```
2601     <input name="receiptDate" type="date" />
2602     <input name="method" value="BANK_TRANSFER" />
2603     <button class="secondary-button" type="submit">登记回款</button>
2604   </form>
2605 </article>
2606 </section>
2607 <section class="panel-grid">
2608   <article class="panel-card">
2609     <header class="panel-head"><h3>预算</h3></header>
2610     <div class="table-shell">
2611       <table>
2612         <thead><tr><th>预算键</th><th>预算额</th><th>已用</th><th>控制模式</th><
2613 /tr></thead>
2614         <tbody>
2615           ${budgets.items
2616             .map(
2617               (item) => `
2618               <tr>
2619                 <td>${item.budget_key}</td>
2620                 <td>${item.budget_amount}</td>
2621                 <td>${item.used_amount}</td>
2622                 <td>${item.control_mode}</td>
2623               </tr>
2624             `
2625             )
2626             .join("")}
2627         </tbody>
2628       </table>
2629     </div>
2630   </article>
2631   <article class="panel-card">
2632     <header class="panel-head"><h3>费用与付款</h3></header>
2633     <div class="table-shell">
2634       <table>
2635         <thead><tr><th>编号</th><th>金额</th><th>状态</th><th>类型</th></tr>
2636 </thead>
2637         <tbody>
2638           ${expenses.items
2639             .slice(0, 4)
2640             .map(
2641               (item) => `
2642               <tr>
2643                 <td>${item.claim_no}</td>
2644                 <td>${item.amount_total}</td>
2645                 <td>${item.status}</td>
2646                 <td>${item.expense_type}</td>
2647               </tr>
2648             `
2649             )
2650             .join("")}
```

```

2651         ${payments.items
2652         .slice(0, 4)
2653         .map(
2654             (item) => `
2655                 <tr>
2656                     <td>${item.payment_no}</td>
2657                     <td>${item.amount_total}</td>
2658                     <td>${item.status}</td>
2659                     <td>${item.source_type}</td>
2660                 </tr>
2661             `
2662         )
2663         .join("")}
2664     </tbody>
2665 </table>
2666 </div>
2667 </article>
2668 </section>
2669 `;
2670 // empty line
2671 root.querySelector("#budget-form").addEventListener("submit", async
2672 (event) => {
2673     event.preventDefault();
2674     const formData = new FormData(event.currentTarget);
2675     try {
2676         await api.createBudget({
2677             budgetYear: Number(formData.get("budgetYear")),
2678             budgetMonth: Number(formData.get("budgetMonth")),
2679             orgUnitId: firstBudget.org_unit_id,
2680             costCenterId: firstBudget.cost_center_id,
2681             budgetAmount: Number(formData.get("budgetAmount")),
2682             budgetKey: formData.get("budgetKey"),
2683         });
2684         app.showSuccess("预算创建成功");
2685         await renderFinance(root, app);
2686     } catch (error) {
2687         app.showError(error);
2688     }
2689 });
2690 // empty line
2691 root.querySelector("#expense-form").addEventListener("submit", async
2692 (event) => {
2693     event.preventDefault();
2694     const formData = new FormData(event.currentTarget);
2695     try {
2696         await api.createExpense({
2697             claimUserId: formData.get("claimUserId"),
2698             costCenterId: formData.get("costCenterId"),
2699             amountTotal: Number(formData.get("amountTotal")),
2700             expenseType: "TRAVEL",

```

```
2701     budgetKey: formData.get("budgetKey"),
2702     attachments: [{ name: "交通票据.pdf" }],
2703   });
2704   app.showSuccess("报销单已提交审批");
2705   await renderFinance(root, app);
2706 } catch (error) {
2707   app.showError(error);
2708 }
2709 });
2710 // empty line
2711 root.querySelector("#payment-form").addEventListener("submit", async
2712 (event) => {
2713   event.preventDefault();
2714   const formData = new FormData(event.currentTarget);
2715   try {
2716     await api.createPayment({
2717       sourceType: formData.get("sourceType"),
2718       sourceId: formData.get("sourceId"),
2719       payeeName: formData.get("payeeName"),
2720       amountTotal: Number(formData.get("amountTotal")),
2721       plannedDate: new Date().toISOString().slice(0, 10),
2722     });
2723     app.showSuccess("付款申请已创建");
2724     await renderFinance(root, app);
2725   } catch (error) {
2726     app.showError(error);
2727   }
2728 });
2729 // empty line
2730 root.querySelector("#receipt-form").addEventListener("submit", async
2731 (event) => {
2732   event.preventDefault();
2733   const formData = new FormData(event.currentTarget);
2734   try {
2735     await api.createReceipt({
2736       contractId: formData.get("contractId"),
2737       amount: Number(formData.get("amount")),
2738       receiptDate: formData.get("receiptDate"),
2739       method: formData.get("method"),
2740     });
2741     app.showSuccess("回款登记成功");
2742     await renderFinance(root, app);
2743   } catch (error) {
2744     app.showError(error);
2745   }
2746 });
2747 }
2748 // section delimiter
2749 apps/web/src/views/workflow.js
2750 import { api } from "../api.js";
```

```

2751 // empty line
2752 function taskActions(task) {
2753   return `
2754     <button class="mini-button" data-action="approve" data-task="$
2755 {task.id}">通过</button>
2756     <button class="mini-button ghost" data-action="reject" data-task="$
2757 {task.id}">驳回</button>
2758 `;
2759 }
2760 // empty line
2761 export async function renderWorkflow(root, app) {
2762   const [templates, todoTasks, doneTasks, initiated] =
2763   await Promise.all([
2764     api.workflowTemplates(),
2765     api.approvalTasks("todo"),
2766     api.approvalTasks("done"),
2767     api.approvalTasks("initiated"),
2768   ]);
2769   root.innerHTML = `
2770     <section class="page-header">
2771       <div>
2772         <p class="eyebrow">WORKFLOW HUB</p>
2773         <h2>流程模板、待办与已办</h2>
2774       </div>
2775     </section>
2776     <section class="panel-grid">
2777       <article class="panel-card">
2778         <header class="panel-head"><h3>待办任务</h3></header>
2779         <div class="task-list">
2780           ${todoTasks
2781             .map(
2782               (task) => `
2783                 <div class="task-card">
2784                   <strong>${task.node_name}</strong>
2785                   <span>${task.biz_type} / ${task.biz_id}</span>
2786                   <em>截至 ${task.due_at}</em>
2787                   <div class="task-actions">${taskActions(task)}</div>
2788                 </div>
2789               `
2790             )
2791             .join("") || `<p class="muted">暂无待办</p>`}
2792         </div>
2793       </article>
2794       <article class="panel-card">
2795         <header class="panel-head"><h3>已办任务</h3></header>
2796         <div class="task-list">
2797           ${doneTasks
2798             .map(
2799               (task) => `
2800                 <div class="task-card done">

```

```

2801         <strong>${task.node_name}</strong>
2802         <span>${task.biz_type} / ${task.biz_id}</span>
2803         <em>动作 ${task.action || "-"}</em>
2804     </div>
2805     ,
2806 )
2807     .join("") || `<p class="muted">暂无已办</p>`
2808 </div>
2809 </article>
2810 </section>
2811 <section class="panel-grid">
2812     <article class="panel-card">
2813         <header class="panel-head"><h3>我的发起</h3></header>
2814         <div class="task-list">
2815             ${initiated
2816             .map(
2817                 (wf) => `
2818                 <div class="task-card">
2819                     <strong>${wf.title}</strong>
2820                     <span>${wf.biz_type} / ${wf.biz_id}</span>
2821                     <em>${wf.status}</em>
2822                 </div>
2823             ,
2824             )
2825             .join("") || `<p class="muted">暂无发起记录</p>`}
2826         </div>
2827     </article>
2828     <article class="panel-card">
2829         <header class="panel-head"><h3>流程模板</h3></header>
2830         <div class="task-list">
2831             ${templates
2832             .map(
2833                 (template) => `
2834                 <div class="task-card">
2835                     <strong>${template.name}</strong>
2836                     <span>${template.biz_type}</span>
2837                     <em>${(template.definition?.nodes ||
2838 [ ]).length} 个节点</em>
2839                 </div>
2840             ,
2841             )
2842             .join("")}
2843         </div>
2844     </article>
2845 </section>
2846 `;
2847 root.querySelectorAll("[data-action]").forEach((button) => {
2848     button.addEventListener("click", async () => {
2849         try {
2850             await api.actTask(button.dataset.task, {

```

```

2851         action: button.dataset.action === "approve" ? "APPROVE" : "REJ
2852 ECT",
2853         comment: button.dataset.action === "approve" ? "页面快速通过" : "页面快
2854 速驳回",
2855     });
2856     app.showSuccess("审批动作已提交");
2857     await renderWorkflow(root, app);
2858     } catch (error) {
2859         app.showError(error);
2860     }
2861 });
2862 });
2863 }
2864 // section delimiter
2865 apps/web/src/views/audit.js
2866 import { api } from "../api.js";
2867 // empty line
2868 export async function renderAudit(root, app) {
2869     const [auditLogs, notifications] = await Promise.all([api.auditLogs(),
2870 api.notifications()]);
2871     root.innerHTML = `
2872         <section class="page-header">
2873             <div>
2874                 <p class="eyebrow">AUDIT & ALERTS</p>
2875                 <h2>审计日志与站内通知</h2>
2876             </div>
2877         </section>
2878         <section class="panel-grid">
2879             <article class="panel-card">
2880                 <header class="panel-head"><h3>审计日志</h3></header>
2881                 <div class="table-shell">
2882                     <table>
2883                         <thead><tr><th>动作</th><th>业务类型</th><th>业务 ID</th><th>请求号</th>
2884 ></tr></thead>
2885                         <tbody>
2886                             ${((auditLogs.items || []).
2887 .slice(0, 10)
2888 .map(
2889     (item) => `
2890         <tr>
2891             <td>${item.action_type}</td>
2892             <td>${item.biz_type}</td>
2893             <td>${item.biz_id}</td>
2894             <td>${item.request_id}</td>
2895         </tr>
2896         `
2897     )
2898 .join("")}
2899                         </tbody>
2900                     </table>

```

```

2901     </div>
2902 </article>
2903 <article class="panel-card">
2904   <header class="panel-head"><h3>通知中心</h3></header>
2905   <div class="task-list">
2906     ${notifications
2907       .map(
2908         (item) => `
2909           <div class="task-card ${item.read_at ? "done" : ""}>
2910             <strong>${item.title}</strong>
2911             <span>${item.content}</span>
2912             <em>${item.biz_ref_type || "SYSTEM"}</em>
2913             $
2914 {item.read_at ? `<span class="muted">已读</span>` : `<button class="mini-b
2915 utton" data-read="${item.id}">设为已读</button>`}
2916           </div>
2917         `
2918       )
2919       .join("")}
2920   </div>
2921 </article>
2922 </section>
2923 `;
2924 root.querySelectorAll("[data-read]").forEach((button) => {
2925   button.addEventListener("click", async () => {
2926     try {
2927       await api.markNotificationRead(button.dataset.read);
2928       app.showSuccess("通知已设为已读");
2929       await renderAudit(root, app);
2930     } catch (error) {
2931       app.showError(error);
2932     }
2933   });
2934 });
2935 }
2936 // section delimiter
2937 packages/shared-contracts/api_contracts.json
2938 {
2939   "name": "enterprise-operations-platform-contracts",
2940   "version": "1.0.0",
2941   "basePath": "/api/v1",
2942   "modules": {
2943     "auth": [
2944       "POST /auth/login",
2945       "POST /auth/refresh",
2946       "GET /users/me"
2947     ],
2948     "workflow": [
2949       "GET /workflows/templates",
2950       "POST /workflows/templates",

```

```

2951         "GET /approvals/tasks",
2952         "POST /approvals/tasks/{taskId}/actions"
2953     ],
2954     "sales": [
2955         "GET /sales/orders",
2956         "POST /sales/orders",
2957         "GET /contracts",
2958         "POST /contracts"
2959     ],
2960     "procurement": [
2961         "GET /procurement/requests",
2962         "POST /procurement/requests",
2963         "GET /procurement/orders",
2964         "POST /procurement/orders"
2965     ],
2966     "inventory": [
2967         "GET /inventory/balances",
2968         "POST /inventory/receipts"
2969     ],
2970     "finance": [
2971         "GET /finance/budgets",
2972         "POST /finance/budgets",
2973         "GET /finance/expenses",
2974         "POST /finance/expenses",
2975         "GET /finance/payment-requests",
2976         "POST /finance/payment-requests",
2977         "GET /finance/receipts",
2978         "POST /finance/receipts"
2979     ],
2980     "dashboard": [
2981         "GET /dashboard/operations"
2982     ],
2983     "audit": [
2984         "GET /audit-logs",
2985         "GET /notifications",
2986         "POST /notifications/{notificationId}/read",
2987         "POST /files/presign"
2988     ]
2989 }
2990 }
2991 // section delimiter
2992 scripts/seed_demo_data.py
2993 """Rebuild the local demo database with fresh seed data."""
2994 // empty line
2995 from __future__ import annotations
2996 // empty line
2997 import json
2998 import sys
2999 from pathlib import Path
3000 // empty line

```